

Snowflake Tutorial

A Complete Guide

27 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/snowflake-tutorial/>

1. Home

Welcome to Snowflake

Snowflake is a cloud-native data platform that provides data warehousing, data lakes, data engineering, data science, and data sharing capabilities — all on a single, fully managed platform. Unlike traditional data warehouses, Snowflake separates compute from storage, allowing them to scale independently.

Key Features

- Elastic compute with Virtual Warehouses
- Separation of storage and compute
- Automatic performance optimization
- Secure data sharing across accounts
- Multi-cloud support (AWS, Azure, GCP)
- Zero-copy cloning and time travel

Course Strategy

This guide takes you from zero to productive with Snowflake. Start with architecture, then create your account and warehouse, learn core objects, load data, run queries, and finally explore external integrations and best practices.

Cloud Services Layer

Authentication, Metadata, Query Optimization

Compute Layer (Virtual Warehouses)

Elastic Clusters — MPP Query Execution

Storage Layer

Compressed Columnar — S3 / Azure Blob / GCS

 **Exercise:** Sign up for a free Snowflake trial account. Explore the web interface and identify the key sections: Worksheets, Databases, Warehouses, and Account settings.

2. Introduction

What is Snowflake?

Snowflake is a fully managed cloud data platform built on a new SQL engine. It supports ANSI SQL and works with structured and semi-structured data (JSON, Avro, Parquet, XML). It runs on AWS, Azure, and GCP.

Cloud Data Warehousing

Unlike traditional DW appliances (Teradata, Netezza) and cloud-first solutions (Redshift, BigQuery), Snowflake's unique architecture separates storage and compute completely, enabling near-infinite scalability and concurrent workload isolation.

Use Cases

- Data warehousing and BI
- Data lakes and data engineering
- Data science and ML
- Data sharing and data marketplaces
- Real-time analytics

 **Exercise:** Write down 3 use cases from your organization or projects where Snowflake could provide value. Consider existing data volumes, query patterns, and concurrency needs.

3. Data Architecture

Three Layers

Layer	Description
Cloud Services	Authentication, query optimization, metadata management, infrastructure management
Compute (Virtual Warehouses)	Elastic clusters of EC2/VM instances that execute queries. Can be resized, suspended, or scaled out.
Storage	Compressed, columnar format in cloud blob storage (S3/Azure Blob/GCS). Pay only for what you use.

Cloud Services Layer

Authentication, Metadata, Query Optimization

Compute Layer (Virtual Warehouses)

Elastic Clusters — MPP Query Execution

Storage Layer

Compressed Columnar — S3 / Azure Blob / GCS

Key Principles

- **Separation of storage and compute** — Scale independently
- **Near-zero management** — No indexes, partitioning, or vacuuming
- **Automatic optimization** — Micro-partitions, clustering, compression

```
-- Snowflake automatically manages storage  
-- No DBA needed for indexing or partitioning  
SHOW DATABASES;
```

 **Exercise:** Draw the Snowflake three-layer architecture from memory. Label each layer and write one sentence about what it does.

4. Functional Architecture

Query Processing

When a query is submitted, Cloud Services parses and optimizes it, then dispatches execution to the Virtual Warehouse. The warehouse scans compressed micro-partitions in parallel using MPP (massively parallel processing).

Concurrency

Multiple warehouses can access the same data simultaneously without lock contention. Each warehouse operates independently with its own compute resources.

Caching

Cache Type	Scope	Duration
Result Cache	Query results	24 hours (or until data changes)
Warehouse Cache	Data files in SSD	While warehouse is running
Metadata Cache	Object metadata	Always fresh

Time Travel

```
-- Query data as of 1 hour ago
SELECT * FROM my_table AT(TIMESTAMP => DATEADD(hour, -1, CURRENT_TIMESTAMP))

-- Query data before a specific statement
SELECT * FROM my_table BEFORE(STATEMENT => '019...');
```


 **Exercise:** Using Snowflake's TIME_TRAVEL feature, query a table as it existed 30 minutes ago. Create a clone of that historical snapshot.

[← Back to Tutorials](#)

5. How to Access

Snowflake Web UI (Snowsight)

The modern web interface for querying, monitoring, and managing your account.

Accessible at <https://<account>.snowflakecomputing.com>.

CLI (SnowSQL)

```
# Install SnowSQL
snowsql -a my_account -u my_user
snowsql> SELECT CURRENT_VERSION();
```

Drivers & Connectors

Language	Driver
Python	snowflake-connector-python
Java	snowflake-jdbc
Node.js	snowflake-sdk

REST API & SDKs

Snowflake provides SQL REST API and SDKs for programmatic access.

 **Exercise:** Install SnowSQL and connect to your Snowflake account. Run `SELECT CURRENT_REGION(), CURRENT_ACCOUNT()`.

[← Back to Tutorials](#)

6. Editions

Edition	Features	Use Case
Standard	Full SQL, time travel (1 day), auto-clustering	Production, BI
Enterprise	+ Multi-cluster, time travel (90 days), data masking	Large-scale, compliance
Business Critical	+ HIPAA, SOC2, GDPR	Healthcare, finance
Virtual Private	+ Isolated compute, dedicated metadata	Government, highest security

```
-- Check your edition
SELECT CURRENT_ACCOUNT_NAME(), CURRENT_REGION();
```

 **Exercise:** Determine which Snowflake edition your account uses.

[← Back to Tutorials](#)

7. Pricing Model

Compute (Credits)

Pay per second of warehouse uptime (min 60s). Credit consumption by size:

Size	Credits/hr	Nodes
X-Small	1	1
Small	2	2
Medium	4	4
Large	8	8

Storage & Serverless

Pay per TB/month for compressed data. Serverless features (auto-clustering, materialized views) consume credits per use.

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME > DATEADD(day, -7, CURRENT_TIMESTAMP);
```

 **Exercise:** Query `WAREHOUSE_METERING_HISTORY` for the last 7 days and estimate cost.

8. Core Objects

Object	Description
Database	Container for schemas and data objects
Schema	Namespace within a database for tables/views/functions
Table	Stores relational data with micro-partitions
View	Logical table based on a query
Stage	Location for files to load/unload (internal or external)
File Format	Describes file types (CSV, JSON, Parquet, etc.)
Function	UDFs, stored procedures (SQL, JavaScript, Python)
Sequence	Generates unique numbers

```
CREATE DATABASE my_db;  
CREATE SCHEMA my_schema;  
CREATE TABLE my_table (id INT, name STRING);  
CREATE STAGE my_stage;  
CREATE FILE FORMAT my_csv TYPE = CSV;
```

 **Exercise:** Create a database, schema, table, stage, and file format in your Snowflake account. Verify each with SHOW commands.

9. Table & View Types

Table Types

Type	Fail-safe	Time Travel	Cost
Permanent	7 days	0-90 days	Standard
Transient	None	0-1 day	Lower
Temporary	None	0-1 day	Session-only

View Types

- **Standard View** — Query definition, no storage
- **Materialized View** — Pre-computed result, refreshed automatically
- **Secure View** — Data hidden from users; predicates applied

```
CREATE TRANSIENT TABLE temp_data (id INT);  
CREATE MATERIALIZED VIEW daily_sales AS  
SELECT DATE_TRUNC('day', sale_ts), SUM(amount)  
FROM sales GROUP BY 1;
```

 **Exercise:** Create a permanent table, a transient table, and a materialized view. Compare the storage costs using ACCOUNT_USAGE views.

10. Login & Setup

1. Go to signup.snowflake.com
2. Choose cloud provider and region
3. Select edition (Standard or Enterprise)
4. Verify email and set password
5. Access Snowsight web UI

```
-- Initial setup: roles and permissions
USE ROLE ACCOUNTADMIN;

CREATE ROLE analyst;
GRANT USAGE ON DATABASE my_db TO ROLE analyst;
GRANT USAGE ON SCHEMA my_db.public TO ROLE analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA my_db.public TO ROLE analyst;

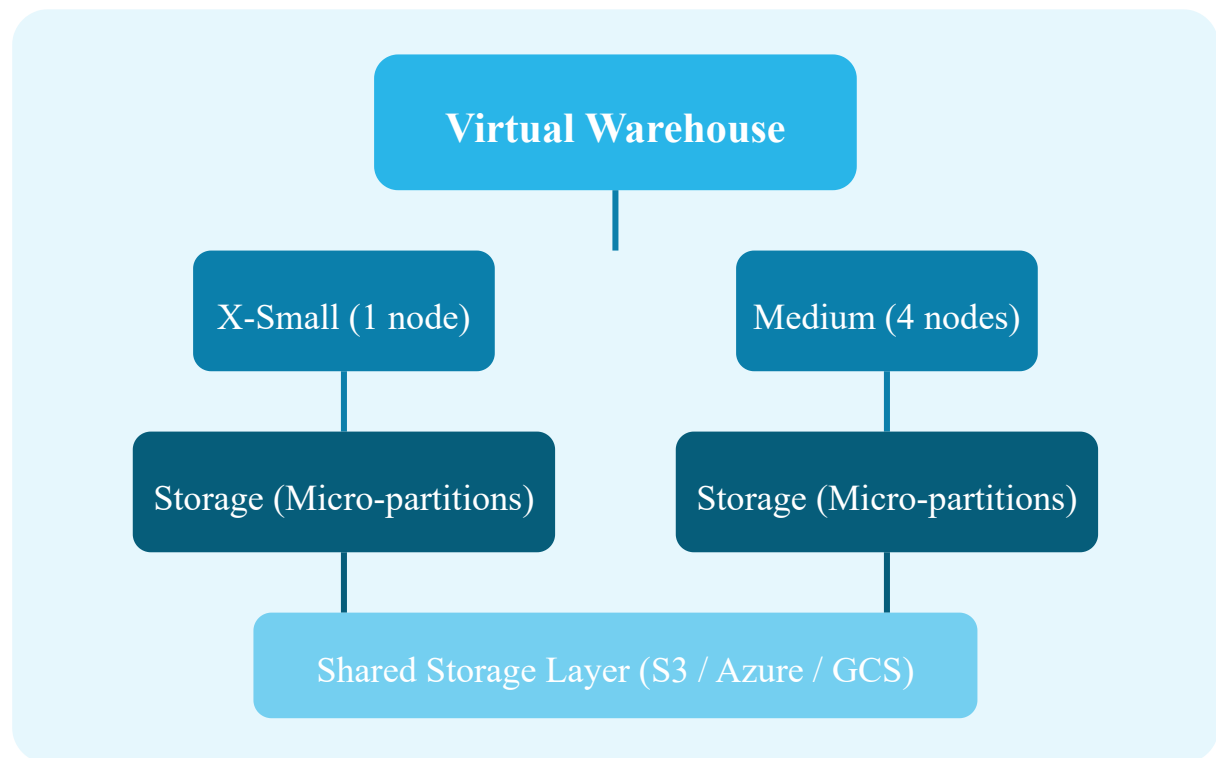
CREATE USER my_analyst PASSWORD = '****' DEFAULT_ROLE = analyst;
GRANT ROLE analyst TO USER my_analyst;
```

Best Practices

- Never use ACCOUNTADMIN for daily work
- Create role hierarchy: SYSADMIN > analyst > reader
- Use network policies for IP restrictions
- Enable MFA

 **Exercise:** Create a new role, user, and grant appropriate permissions. Log in as that user and verify you can query the database.

11. Virtual Warehouse



Manage Warehouses

```
-- Create
CREATE WAREHOUSE my_wh
  WITH WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 300
  AUTO_RESUME = TRUE
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 3
  SCALING_POLICY = 'ECONOMY';

-- Resize
ALTER WAREHOUSE my_wh SET WAREHOUSE_SIZE = 'SMALL';

-- Suspend / Resume
ALTER WAREHOUSE my_wh SUSPEND;
ALTER WAREHOUSE my_wh RESUME;
```

Multi-Cluster Warehousing

Available in Enterprise edition. Automatically adds clusters when queuing occurs, up to MAX_CLUSTER_COUNT.

Best Practices

- Set AUTO_SUSPEND to 5-10 min for cost savings
- Use separate warehouses for different workloads
- Right-size: start X-Small, test, scale up

 **Exercise:** Create a multi-cluster warehouse (XS, min 1, max 3). Run concurrent queries and monitor cluster usage via ADMIN.

[← Back to Tutorials](#)

12. Database

```
-- Create
CREATE DATABASE sales_db;

-- Clone (zero-copy)
CREATE DATABASE sales_dev_db CLONE sales_db;

-- Drop
DROP DATABASE IF EXISTS sales_dev_db;

-- Undrop (time travel recovery)
UNDROP DATABASE sales_dev_db;

-- Set data retention
ALTER DATABASE sales_db SET DATA_RETENTION_TIME_IN_DAYS = 30;
```

Zero-Copy Cloning

Creates a copy of the database that shares the underlying storage until changes are made. No additional storage cost for the clone until you modify data.

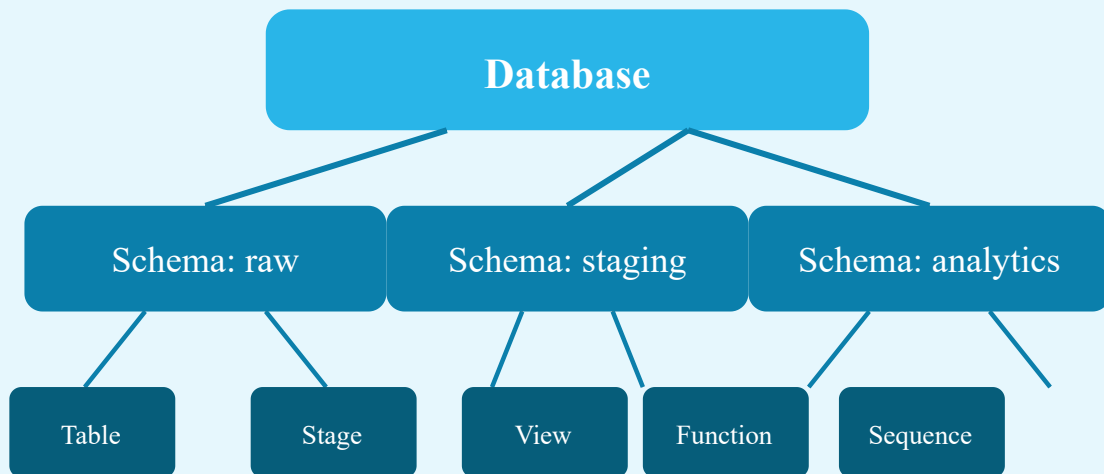
Sharing

Create a share for secure data sharing between accounts:

```
CREATE SHARE sales_share;
GRANT USAGE ON DATABASE sales_db TO SHARE sales_share;
ALTER SHARE sales_share ADD ACCOUNTS = another_account;
```

 **Exercise:** Create a database, clone it, verify the clone is identical. Drop the clone, then undrop it.

13. Schema



Snowflake Object Hierarchy

```
-- Create
CREATE SCHEMA sales_db.raw;

-- Create with managed access
CREATE SCHEMA sales_db.staging WITH MANAGED ACCESS;

-- Clone
CREATE SCHEMA sales_db.analytics CLONE sales_db.raw;

-- Set retention
ALTER SCHEMA sales_db.raw SET DATA_RETENTION_TIME_IN_DAYS = 15;

-- List schemas
SHOW SCHEMAS IN DATABASE sales_db;
```

Schema-Level Objects

- Tables
- Views
- Stages
- File Formats
- Functions
- Stored Procedures
- Sequences

Managed Access Schemas

Only the schema owner can grant privileges on objects within the schema. Useful for secure data sharing.

 **Exercise:** Create three schemas in a database: raw, staging, analytics. Clone raw to staging and verify the clone.

14. Table & Columns

Data Types

Category	Types
Numeric	NUMBER, INT, FLOAT, DECIMAL
String	VARCHAR, CHAR, TEXT, STRING
Date/Time	DATE, TIME, TIMESTAMP, TIMESTAMP_LTZ, TIMESTAMP_NTZ, TIMESTAMP_TZ
Semi-structured	VARIANT, OBJECT, ARRAY
Binary	BINARY

```
CREATE TABLE employees (  
  emp_id INT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  salary NUMBER(10,2),  
  hire_date DATE DEFAULT CURRENT_DATE,  
  metadata VARIANT,  
  dept_id INT,  
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);  
  
-- Add/drop columns  
ALTER TABLE employees ADD COLUMN email VARCHAR(255);  
ALTER TABLE employees DROP COLUMN email;  
  
-- Set default  
ALTER TABLE employees ALTER COLUMN salary SET DEFAULT 0;
```

Constraints

Snowflake supports PRIMARY KEY, UNIQUE, FOREIGN KEY, and NOT NULL. These are informational (not enforced) except NOT NULL.

 **Exercise:** Create tables with various data types including VARIANT. Insert JSON data into the VARIANT column.

15. Loading Data

Using COPY INTO

```
-- Create stage and file format
CREATE OR REPLACE STAGE my_stage;
CREATE OR REPLACE FILE FORMAT csv_ff
  TYPE = CSV
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '';

-- Stage a file
PUT file://data/sales.csv @my_stage;

-- Copy into table
COPY INTO sales_table
FROM @my_stage/sales.csv
FILE_FORMAT = csv_ff
ON_ERROR = 'CONTINUE';
```

Validation

```
-- Validate before loading
COPY INTO sales_table
FROM @my_stage/sales.csv
FILE_FORMAT = csv_ff
VALIDATION_MODE = RETURN_ERRORS;

-- View history
SELECT * FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME=>'sales_table',
  START_TIME=>DATEADD(hours, -24, CURRENT_TIMESTAMP())));
```


Auto-Ingest (Snowpipe)

Snowpipe automates loading when new files arrive in a stage:

```
CREATE PIPE sales_pipe AS  
  COPY INTO sales_table FROM @my_stage FILE_FORMAT = csv_ff;  
ALTER PIPE sales_pipe REFRESH;
```

 **Exercise:** Upload a CSV file to a stage and load it into a table. Use `VALIDATION_MODE` first, then load. Verify with `COPY_HISTORY`.

16. Useful Queries

Time Travel

```
-- Query at a specific timestamp
SELECT * FROM my_table
  AT(TIMESTAMP => '2025-01-01 12:00:00'::TIMESTAMP);

-- Query before a DML statement
SELECT * FROM my_table
  BEFORE(STATEMENT => '019abc123...');

-- Create clone from history
CREATE TABLE restored_table CLONE my_table
  AT(OFFSET => -3600);  -- 1 hour ago
```

Query Metadata

```
-- Show tables
SHOW TABLES IN SCHEMA my_db.public;

-- Describe table
DESC TABLE my_table;

-- Query history
SELECT * FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
  WHERE QUERY_TYPE = 'SELECT'
  ORDER BY START_TIME DESC
  LIMIT 10;
```

Sampling

```
-- Random sample (0.1%)
SELECT * FROM my_table SAMPLE (0.1);

-- Row count approximation
SELECT APPROX_COUNT_DISTINCT(customer_id) FROM sales;
```

 **Exercise:** Use TIME TRAVEL to query data from 1 hour ago. Create a clone of that snapshot. Run query history to review recent queries.

[← Back to Tutorials](#)

17. Monitoring Usage

Account Usage Views

Query `SNOWFLAKE.ACCOUNT_USAGE` schema:

```
-- Warehouse metering
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY;

-- Storage usage
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.STORAGE_USAGE;

-- Query history
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
  WHERE DATABASE_NAME = 'SALES_DB';

-- Login history
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY;

-- Auto-cluster history
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.AUTOMATIC_CLUSTERING_HISTORY;
```

INFORMATION_SCHEMA

```
SELECT * FROM INFORMATION_SCHEMA.TABLES;
SELECT * FROM INFORMATION_SCHEMA.COLUMNS;
SELECT * FROM INFORMATION_SCHEMA.STAGES;
```

Monitoring Best Practices

- Set up resource monitors to cap spending
- Use email notifications for threshold alerts
- Monitor query performance with QUERY_HISTORY

- Track storage trends with STORAGE_USAGE

```
-- Create resource monitor
CREATE RESOURCE MONITOR daily_cap
  WITH CREDIT_QUOTA = 100
  FREQUENCY = DAILY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS ON 80% DO NOTIFY
           ON 100% DO SUSPEND;

ALTER WAREHOUSE my_wh SET RESOURCE_MONITOR = daily_cap;
```

 **Exercise:** Create a resource monitor with 50-credit daily quota and a suspend trigger at 100%. Apply it to a warehouse.

18. Caching

Result Cache

Query results are cached for 24 hours (or until underlying data changes). Identical queries served from cache — no warehouse needed.

```
-- First run: executes (billed)
SELECT COUNT(*) FROM sales;

-- Second run: served from result cache (free, instant)
SELECT COUNT(*) FROM sales;
```

Warehouse Cache

Data files cached in SSD on warehouse nodes. Persists while warehouse is running. If warehouse is suspended, cache is lost.

Metadata Cache

Object metadata (row count, column stats, partitions) is always cached and served instantly for optimization.

Best Practices

- Keep warehouses running for repeated queries to benefit from cache
- Use `RESULT_SCAN` to access previous results

```
SELECT * FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

 **Exercise:** Run the same query twice. Compare execution times using `QUERY_HISTORY`. Note which run used cached results.

19. Unload to Local

COPY INTO Location

```
-- Export table to stage files
COPY INTO @my_stage/unload/
FROM sales_table
FILE_FORMAT = (TYPE = CSV COMPRESSION = GZIP)
HEADER = TRUE
OVERWRITE = TRUE;

-- Export specific columns
COPY INTO @my_stage/unload/
FROM (SELECT id, name, amount FROM sales_table)
FILE_FORMAT = (TYPE = PARQUET);

-- Download from stage
GET @my_stage/unload/ file://local_output/;
```

Export Formats

Format	Use Case
CSV	Universal, spreadsheet-compatible
JSON	Web applications, APIs
Parquet	Columnar, efficient, data pipelines
ORC	Hive/Spark compatibility

Partitioned Export

```
COPY INTO @my_stage/unload/  
FROM sales_table  
PARTITION BY ('year=' || YEAR(sale_date))  
FILE_FORMAT = (TYPE = PARQUET);
```

 **Exercise:** Export a table as CSV with headers, download using GET, and verify file contents.

[← Back to Tutorials](#)

20. Load from S3

External Stage

```
-- Create external stage for S3
CREATE OR REPLACE STAGE s3_stage
  URL = 's3://my-bucket/snowflake-data/'
  STORAGE_INTEGRATION = s3_integration
  FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1);

-- List files
LIST @s3_stage;

-- Load from external stage
COPY INTO my_table
FROM @s3_stage/
PATTERN = '.*\.csv'
ON_ERROR = 'SKIP_FILE';
```

AWS Integration

```
-- Create storage integration (ACCOUNTADMIN)
CREATE STORAGE INTEGRATION s3_integration
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = S3
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456:role/snowflake-role'
  STORAGE_ALLOWED_LOCATIONS = ('s3://my-bucket/');

DESC STORAGE INTEGRATION s3_integration;
-- Use the AWS_IAM_USER_ARN and EXTERNAL_ID to set up trust
```

Loading from Azure / GCP

Similar patterns for Azure Blob (AZURE_INTEGRATION) and GCS (STORAGE_PROVIDER = GCS).

 **Exercise:** Set up an external S3 stage and load data from a public S3 bucket. Verify data with SELECT.

21. Unload to S3

```
-- Unload to external stage
COPY INTO @s3_stage/unload/
FROM my_table
FILE_FORMAT = (TYPE = PARQUET COMPRESSION = SNAPPY)
OVERWRITE = TRUE;

-- Partitioned unload
COPY INTO @s3_stage/unload/
FROM my_table
PARTITION BY ('dt=' || DATE(sale_date))
FILE_FORMAT = (TYPE = CSV)
HEADER = TRUE;

-- Validate unload
SELECT COUNT(*) FROM @s3_stage/unload/;
```

Unload Patterns

Pattern	When to Use
CSV with header	Data sharing, BI tools
Parquet snappy	Efficient columnar, downstream ETL
JSON	API consumption
Partitioned	Hive-style partitioning for Spark/Athena

Best Practices

- Use `OVERWRITE = TRUE` for idempotent exports
- Add `SINGLE = TRUE` to get one file (not parallel)

- Set MAX_FILE_SIZE for file size control

```
COPY INTO @s3_stage/report.csv
FROM (SELECT * FROM my_table ORDER BY id)
FILE_FORMAT = (TYPE = CSV HEADER = TRUE)
SINGLE = TRUE
MAX_FILE_SIZE = 500000000;
```

 **Exercise:** Unload a table to S3 in Parquet format with Snappy compression. Verify the files in S3.

[← Back to Tutorials](#)

22. Resources

Quick Guide

Task	Command
List databases	SHOW DATABASES;
Use database	USE DATABASE my_db;
List schemas	SHOW SCHEMAS IN my_db;
List tables	SHOW TABLES IN my_db.my_schema;
Describe table	DESC TABLE my_table;
List stages	SHOW STAGES;
List warehouses	SHOW WAREHOUSES;
Show current role	SELECT CURRENT_ROLE();
Show current warehouse	SELECT CURRENT_WAREHOUSE();

Useful References

- [Official Documentation](#)
- [Snowflake Community](#)
- [Quickstarts](#)

23. Quick Guide

Snowflake SQL Cheatsheet

Category	SQL
Warehouse	<pre>CREATE WAREHOUSE wh WAREHOUSE_SIZE = 'XSMALL' AUTO_SUSPEND = 300;</pre>
Database	<pre>CREATE DATABASE db; CREATE DATABASE db2 CLONE db;</pre>
Schema	<pre>CREATE SCHEMA db.schema;</pre>
Table	<pre>CREATE TABLE t (id INT, name STRING);</pre>
Stage	<pre>CREATE STAGE s; PUT file://data.csv @s;</pre>
Load	<pre>COPY INTO t FROM @s FILE_FORMAT=(TYPE=CSV);</pre>
Unload	<pre>COPY INTO @s/out/ FROM t; GET @s/out/ file://local/;</pre>
External	<pre>CREATE STAGE s3 URL='s3://bucket/' STORAGE_INTEGRATION=i;</pre>
Time Travel	<pre>SELECT * FROM t AT(TIMESTAMP=>...);</pre>
Clone	<pre>CREATE TABLE t2 CLONE t;</pre>

Account Management

Task	Command
List users	SHOW USERS;
List roles	SHOW ROLES;
Set role	USE ROLE ACCOUNTADMIN;
Resource monitor	CREATE RESOURCE MONITOR m WITH CREDIT_QUOTA=100;

 **Exercise:** Use this cheatsheet for your next Snowflake session. Try to complete a full ETL cycle using only the commands listed.

24. Useful Resources

Documentation & Learning

- **Snowflake Docs:** docs.snowflake.com — Comprehensive SQL reference, guides, and release notes
- **Snowflake Quickstarts:** quickstarts.snowflake.com — Hands-on labs
- **Snowflake University:** Free training and certification prep

Community

- **Snowflake Community:** community.snowflake.com — Forums, ideas, knowledge base
- **Stack Overflow:** Tagged questions for troubleshooting
- **GitHub:** snowflakedb organization — SDKs, connectors, samples

Tools & Ecosystem

- **dbt:** Data transformation tool with native Snowflake adapter
- **Airflow:** Orchestration with Snowflake operators
- **Tableau / Power BI:** BI tools with Snowflake connectors
- **Snowpark:** DataFrame API for Python/Java/Scala

25. Discussion

Q&A

Q: Is Snowflake a data warehouse or a data lake?

A: Both. It's a data platform that supports warehousing workloads (structured, BI) and lake workloads (semi-structured, ELT).

Q: How does Snowflake compare to Redshift / BigQuery?

A: Snowflake offers better concurrency isolation via separate warehouses, easier management (no tuning), and cross-cloud portability. Redshift is cheaper at very large scale. BigQuery has auto-scaling but less predictable pricing.

Q: When should I use Snowpipe vs COPY INTO?

A: Use Snowpipe for continuous, near-real-time ingestion. Use COPY INTO for batch/scheduled loads.

Q: Does Snowflake charge for queries that hit result cache?

A: No. Result cache queries don't consume warehouse credits.

Further Learning

- SnowPro Core certification
- Advanced: Snowpark, Streams & Tasks
- Data sharing and Data Marketplace
- Snowflake for data science (ML, feature stores)

26. Appendix

Best Practices

- Use separate warehouses for dev, test, production
- Set AUTO_SUSPEND aggressively (5-10 min)
- Use transient tables for staging data
- Leverage zero-copy cloning for dev/test environments
- Cluster large tables on frequently filtered columns
- Use materialized views for dashboards
- Set up resource monitors on all warehouses

Resume Builder

Highlight these skills on your resume: Snowflake architecture, Virtual Warehouses, Time Travel, Zero-Copy Cloning, Snowpipe, Data Sharing, Snowpark, Performance Tuning, Security (RBAC, network policies, data masking).

HR Interview Questions

- What is Snowflake and how does it differ from traditional data warehouses?
- Explain the three-layer architecture.
- How does Snowflake handle concurrency?
- What is a Virtual Warehouse and how do you size it?
- Explain Time Travel and Fail-safe.
- How do you load data into Snowflake?
- What editions does Snowflake offer?

Glossary

Term	Definition
Virtual Warehouse	Elastic compute cluster for query execution
Micro-partition	Smallest storage unit (50-500 MB compressed)
Time Travel	Access historical data within retention period
Fail-safe	7-day disaster recovery period (Snowflake managed)
Zero-Copy Clone	Instant copy without duplicating storage
Snowpipe	Automated continuous data ingestion

27. Bonus: Who's Who in Snowflake Ecosystem

Person / Entity	Role
Benoit Dageville	Co-founder & President of Products — Snowflake's database architecture visionary
Thierry Cruanes	Co-founder & CTO — Designed Snowflake's multi-cluster architecture
Marcin Zukowski	Co-founder — Early architect of Snowflake's cloud-native engine
Sridhar Ramaswamy	CEO (since 2024) — Former SVP AI at Google, leading AI/data convergence
Frank Sloatman	Former CEO (2019-2024) — Led IPO and growth to \$5B+ revenue
Christian Kleinerman	EVP Product — Oversees product strategy, Snowpark, governance
Snowflake (SNOW)	Public company (NYSE: SNOW) — \$80B+ market cap, 10K+ customers
Snowflake Ventures	Investment arm — Backing data/AI startups (dbt, Monte Carlo, etc.)
Snowflake Community	User ecosystem — 500K+ members, 100+ user groups globally

Key Partners

- **AWS / Azure / GCP** — Cloud infrastructure providers
- **Fivetran / Airbyte** — ELT pipeline partners
- **dbt Labs** — Transformation layer

- **Tableau / Looker / Power BI** — BI and visualization
- **Informatica / Talend** — Enterprise data integration